



Universidade Federal do Cariri
Centro de Ciências e Tecnologia

Relatório Técnico - Sistema de Tráfego Urbano

Aplicações práticas do Uso de Processos e Threads em C no contexto
de Sistemas Operacionais

Gabriel Souza Santos

José Dhonatan Fernandes de Almeida

Letícia Maria dos Santos Dias

Sarah Mendes Teles

Sumário

1. Motivação.....	2
2. Decisões de implementação.....	2
2.1. Mapa.....	2
2.1.1. mapa.h	4
2.1.2. mapa.c	6
2.2. Threads	8
2.2.1. Threads dos veículos.....	8
2.2.2. Thread do relógio	9
2.2.2.1. Sincronização das threads trabalhadoras.....	9
2.2.4. Thread do analisador	9
2.2.5. Thread do renderizador	10
2.3. Mecanismos de sincronização.....	11
2.3.1. Sincronização pelo Clock.....	12
2.3.2 Sincronização entre Threads	12
2.4. Ausência de espera ocupada	13
2.4.1. Espera por Eventos em vez de Verificação Contínua	13
2.4.2. Uso de Mutex para Controle de Concorrência	14
2.5. Estratégia contra deadlock.....	14
2.5.1. Centralização das decisões no Analyser	14
2.5.2. Uso de um único mutex para recursos compartilhados.....	15
2.5.3. Sincronização por ticks.....	16
2.5.4. Validações antes do acesso ao recurso compartilhado.....	16
2.6. Ambulância	16
2.6.1. Controle da Coordenada de Prioridade.....	17
2.6.2. Atualização da Prioridade.....	17
2.6.3. Consulta da Prioridade.....	18
2.6.4. Integração com o Controlador de Semáforos.....	18
2.6.5. Sobrescrição do Ciclo Normal.....	18
2.6.6. Segurança da implementação.....	19
3. Membros e responsabilidades.....	20
4. Conclusão.....	20

1. Motivação

Reconhecendo a importância da programação concorrente e paralela e visando o aprendizado correto dos conceitos estudados ao longo da disciplina de Sistemas Operacionais. O seguinte trabalho consiste no desenvolvimento de um sistema de tráfego urbano que utiliza processos e threads em C com o objetivo de construir uma aplicação sem espera ocupada, sem deadlocks e com uso de mutexes e semáforos na prática. No presente relatório, destaco quais foram as decisões tomadas pela equipe no momento da implementação, nossos objetivos e resultados obtidos.

2. Decisões de Implementação

2.1. Mapa

```
v < < d < ! < <
v # # v ! # # ^
! # # ! ^ # # ^
v ! < . . ! < l
r > ! . . > ! ^
v # # v ! # # !
! # # ! ^ # # ^
v ! < . . ! < l
r > ! . . > ! ^
v # # v ! # # !
v # # ! ^ # # ^
> > ! > u > > ^
```

A primeira escolha que tivemos que fazer ao iniciar o desenvolvimento foi como estruturar o mapa e a solução acima foi a que escolhemos, consideramos cada tipo de caractere como um tipo de objeto do mundo real, seja estrada, semáforo ou casa. Na tabela a seguir temos a representação do significado de cada símbolo:

Para que os veículos não batam nos obstáculos implementamos a função

Símbolo	Significado
^	Via com sentido para cima
v	Via com sentido para baixo
<	Via com sentido para a esquerda
>	Via com sentido para a direita
.	Via sem direção definida, normalmente utilizada em cruzamentos ou interseções
!	Área de espera ou parada(Semáforo)
#	Obstáculo ou parede
u	Giro de direção opcional para cima
d	Giro de direção opcional para baixo
l	Giro de direção opcional para esquerda
r	Giro de direção opcional para direita

`fill_tiles()` que reconhece apenas esses caracteres como válidos: '^', 'v', '<', '>', '.', '!', 'u', 'd', 'l', 'r'. Dessa forma o caractere # bem como qualquer outro que fugisse à regra entra no `default`:

default:

```
tile->type = TILE_BLOCKED;  
tile->is_occupied = true;  
break;
```

Como `#` não é um dos caracteres válidos, ele é convertido para `TILE_BLOCKED` e `is_occupied = true` funcionando de fato com uma barreira, um espaço sempre ocupado.

Vale ressaltar que no mapa temos oito cruzamentos e semáforos para que os veículos simulem a passagem por eles evitando os principais problemas que estudamos em sala de aula que são os deadlocks e a espera ocupada, que aprofundaremos em tópicos posteriores.

2.1.1. mapa.h

O arquivo `map.h` definido no diretório `include` traz uma definição da interface pública do mapa e sua manipulação de coordenadas.

Observações importantes:

Uma célula bloqueada (blocked) é uma célula na qual veículos não podem transitar e que é permanentemente tratada como ocupada pelo sistema. Células que não são bloqueadas podem estar ocupadas pela presença de um veículo ou permanecer livres para circulação.

Funções do `map.h`

`map_new()` : Essa função aloca recursos, inicializa recursos internos e carrega o mapa.

- O mapa é construído em memória a partir de um arquivo de configuração o `map.txt`.
- Conforme os símbolos definidos em `TileType`. O carregamento é realizado em duas passagens pelo arquivo: a primeira determina as dimensões da malha e a segunda preenche as células do mapa.
- Qualquer caractere desconhecido é tratado como `TILE_BLOCKED` e marcado como ocupado, tornando a célula permanentemente intransitável, como já mencionado.

`map_get_tile_type()`: Obtém o tipo de uma célula numa dada coordenada.

Utiliza como parâmetros um ponteiro (*map) para a estrutura do mapa e a estrutura de coordenada contendo as posições X e Y desejadas.

A função retorna o valor correspondente do enum TileType. Se o mapa for inválido ou a coordenada estiver fora dos limites do mapa, retorna TILE_BLOCKED. A linha abaixo ilustra os detalhes mencionados:

```
TileType map_get_tile_type(const Map *map, Coord position);
```

map_is_blocked(): Checa se uma célula é permanentemente intransitável.

Esta função verifica apenas o tipo da célula, não considerando seu estado de ocupação. Assim, uma célula ocupada por um veículo continua sendo considerada transitável caso seu tipo seja diferente de TILE_BLOCKED.

map_is_occupied(): Checa se uma célula está ocupada no momento por um veículo ou obstáculo físico. Esta verificação realiza uma leitura thread-safe utilizando o mecanismo interno de sincronização do mapa. Células são consideradas ocupadas se houver um veículo nelas ou se forem do tipo TILE_BLOCKED.

Retorna true Se a célula estiver indisponível para tráfego imediato ou false, se a célula estiver livre e apta a receber um veículo.

```
bool map_is_occupied(Map *map, Coord position);
```

map_transfer_occupant(): Realiza a transferência de um ocupante entre duas células, verifica se a célula de destino é válida, não está bloqueada e não se encontra ocupada. Caso a movimentação seja permitida, a ocupação é removida da célula de origem e é atribuída à célula de destino como uma operação atômica protegida pelo mecanismo interno de sincronização do mapa.

map_reserve_spawn_point(): Procura uma posição inicial disponível para um novo veículo e a reserva. Varre a malha viária em busca de uma célula pertencente a uma via direcional válida(TILE_ROAD_UP, TILE_ROAD_DOWN, TILE_ROAD_LEFT

ou `TILE_ROAD_RIGHT`) que não esteja ocupada. Ao encontrar uma posição adequada, altera seu estado para ocupado e retorna sua coordenada.

Obs.: Esta função é projetada para ser utilizada exclusivamente durante a criação de um veículo.

2.1.2. `map.c`

O arquivo `map.c` faz a implementação da lógica completa do mapa, esse arquivo está no diretório `src` e as suas funções são:

`fill_tiles()`: Responsável por preencher as células do mapa a partir do conteúdo do arquivo. Percorre o arquivo aberto, ignorando quebras de linha e espaços, mapeando cada caractere válido para o `TileType` correspondente. Caracteres não reconhecidos são convertidos para `TILE_BLOCKED` e marcados imediatamente como ocupados, tornando a célula permanentemente intransitável.

`map_new()`: Implementação da construção do mapa a partir de arquivo. O carregamento ocorre em duas passagens pelo arquivo: primeiro calcula as dimensões do mapa com `find_dimensions()`, depois aloca a estrutura e o array de tiles com `calloc()`, garantindo o estado padrão (`TILE_BLOCKED` e `is_occupied = false`). Por fim, popula o grid com `fill_tiles()`.

Detalhe:

- Caracteres inválidos encontrados durante a leitura são tratados como `TILE_BLOCKED` e marcados como ocupados.
- Falhas em qualquer etapa de alocação resultam em limpeza manual dos recursos já adquiridos e retorno de `NULL`.

`map_get_tile_type()`: Implementação da consulta de tipo de tile.

Realiza verificação de limites com `map_is_within_bounds()`. Caso a coordenada esteja fora dos limites do mapa ou o ponteiro do mapa seja `NULL`, retorna

TILE_BLOCKED. Caso contrário, delega para *tile_at()* a obtenção da célula correspondente e retorna seu tipo.

4. *map_is_blocked()*: Implementação da verificação de célula bloqueada.

Verifica apenas o tipo da célula, retornando verdadeiro quando *tile->type == TILE_BLOCKED*. Não considera o estado de ocupação por veículos. Essa consulta é feita sem necessidade de lock, pois o campo *type* é imutável após a construção do mapa, sendo definido apenas em *fill_tiles()*.

5. *map_is_occupied()*: Implementação da verificação de ocupação. Valida os limites antes de qualquer acesso (retornando true para posições fora do mapa, por segurança). Em seguida, utiliza o mecanismo interno de sincronização do mapa para ler *tile->is_occupied* de forma thread-safe.

Uma célula é considerada ocupada quando há um veículo nela ou quando o seu tipo é *TILE_BLOCKED*.

6. *map_transfer_occupant()*: Implementação da transferência de ocupação entre duas células.

Adquire o mutex global do mapa antes de inspecionar ou alterar qualquer tile, garantindo que a verificação de disponibilidade do destino e a atualização das células de origem e destino ocorram como uma operação atômica. Isso evita **condições de corrida** em que múltiplos veículos poderiam observar a mesma célula de destino como livre simultaneamente.

A transferência só ocorre se o destino não for *TILE_BLOCKED* e não estiver ocupado; caso contrário, nenhum estado é alterado.

7. *map_reserve_spawn_point()*: Implementação da busca e reserva de ponto de spawn.

Mantém o mutex global do mapa adquirido durante toda a varredura linear do grid (linha por linha), retornando antecipadamente (com unlock) uma célula de via

direcional válida (TILE_ROAD_UP, TILE_ROAD_DOWN, TILE_ROAD_LEFT ou TILE_ROAD_RIGHT) que esteja livre.

- As células TILE_ROAD genéricas e TILE_WAIT não são consideradas pontos válidos de spawn.
- Caso nenhuma célula adequada seja encontrada, libera o mutex e retorna NULL_COORD.

Observação importante: A busca é sequencial e sem aleatoriedade; portanto, os primeiros pontos do grid tendem a ser utilizados antes dos demais.

2.2. Threads

No projeto utilizamos threads para representar os veículos, uma para o relógio, uma para o analisador e uma para o renderizador. Abaixo estão as threads identificadas e o papel de cada uma.

2.2.1. Threads dos veículos

Presente em `src/vehicle.c` a função que executa como thread de veículos é a seguinte:

```
void *vehicle_update(void *vehicle_args)
```

vehicle_args: Estrutura contendo o veículo e os recursos compartilhados necessários para a execução da thread.

Ela representa o ciclo de vida de um veículo durante a simulação. Cada veículo criado pelo programa executa essa função em uma thread independente. Durante cada tick da simulação, ela:

- Verifica se o veículo deve se mover conforme sua velocidade;
- Calcula a posição desejada;
- Valida se o movimento é permitido;
- Envia uma requisição ao analisador;

- Aguarda a decisão do analisador (dorme enquanto espera);
- Atualiza sua posição e direção caso o movimento seja aprovado;
- Sincroniza-se com o relógio para iniciar o próximo tick.

Cada veículo da simulação executa esta rotina em uma thread independente. A função permanece ativa durante toda a duração da simulação, sincronizando suas ações com o relógio global. A cada tick, determina se o veículo pode se mover conforme sua velocidade, calcula o destino pretendido, realiza validações locais de movimentação, envia a solicitação ao analisador e, caso aprovada, atualiza sua posição e direção no mapa.

A sincronização entre veículos é realizada indiretamente por meio do analisador e do mapa, evitando conflitos de ocupação entre múltiplas threads concorrentes. Além disso, vale ressaltar que as threads dos veículos concorrem diretamente com as do semáforo quando a próxima célula é uma do tipo `TILE_WAIT` (célula de espera).

2.2.2. Thread do relógio (Clock)

A função `clock_update()` presente em `src/clock.c` implementa o laço principal da thread responsável pelo relógio global da simulação. Sua função é coordenar o avanço dos ticks e sincronizar todas as threads trabalhadoras.

A cada iteração, a thread permanece bloqueada na **variável de condição** `cond_clock` enquanto nem todas as threads trabalhadoras tiverem sinalizado a conclusão do tick atual. Quando todas finalizam seu processamento, o relógio realiza a troca dos buffers do analisador e do do semáforo (para salvar os estado anterior da simulação), reinicializa o contador de trabalhadores concluídos, incrementa o tick da simulação, aguarda um pequeno intervalo para controlar a velocidade da execução e, por fim, desperta simultaneamente todas as threads trabalhadoras por meio de um `pthread_cond_broadcast()`.

A espera é implementada utilizando um laço `while`, garantindo que despertares das variáveis de condição não comprometam a sincronização entre as threads.

2.2.2.1. Sincronização das threads trabalhadoras

A função *clock_signal* é utilizada pelas threads trabalhadoras ao término do processamento de cada tick da simulação. Ao ser chamada, incrementa de forma **protegida pelo mutex** o número de threads que concluíram suas atividades. Quando a última thread trabalhadora finaliza o tick, a função sinaliza a variável de condição *cond_clock*, despertando a thread do relógio para que o próximo tick possa ser iniciado.

Após essa sinalização, cada thread trabalhadora permanece bloqueada na variável de condição *cond_workers* enquanto o relógio não avançar para o próximo tick. Dessa forma, todas as threads aguardam sincronizadas **sem realizar espera ocupada**, retomando a execução apenas quando o relógio libera um novo ciclo da simulação.

2.2.3. Thread do analisador

A função *analyser_update()* presente no arquivo *src/analyser.c*, implementa o laço principal da thread responsável pelo analisador de movimentos da simulação. Sua finalidade é atuar entre as threads dos veículos e o estado compartilhado do mapa, garantindo que as movimentações ocorram de forma consistente e livre de conflitos.

Durante cada tick da simulação, a thread permanece bloqueada até que todos os veículos tenham enviado suas respectivas requisições de movimento. Essa sincronização é realizada por meio da variável de condição *analyser_cond* e do contador *pending_count*, assegurando que a análise só seja iniciada quando todas as solicitações do tick atual estiverem disponíveis.

Após receber todas as requisições, a thread percorre cada slot de requisição, verificando inicialmente se o veículo permanecerá na mesma posição. Em seguida, controla conflitos de destino utilizando uma matriz temporária (*destinations*), impedindo que dois veículos recebam aprovação para ocupar a mesma célula durante o mesmo tick.

Quando o destino ainda não foi reservado, a thread solicita ao módulo do mapa a transferência física do ocupante por meio da função *map_transfer_occupant()*. Caso a movimentação seja aceita, o destino é marcado como ocupado para o restante da análise e a requisição recebe o status de aprovada; caso contrário, a movimentação é negada.

Concluída a análise, a thread desperta individualmente cada veículo utilizando a variável de condição associada ao seu slot exclusivo, permitindo que cada thread leia o resultado de sua requisição e prossiga com a execução.

Ao final de cada ciclo, a thread sincroniza-se com o relógio global por meio da função *clock_signal()*, permanecendo bloqueada até o início do próximo tick da simulação.

2.2.4. Thread do renderizador

A thread do renderizador é responsável por exibir, a cada tick da simulação, o estado anterior dos veículos e dos semáforos no terminal. Sua execução ocorre de forma assíncrona com as demais threads por meio do relógio global, garantindo que a visualização mostre todos os estados do sistema.

A função principal da thread é *render_update()*, que recebe uma estrutura *RenderArgs* contendo ponteiros para o relógio, o mapa, o renderizador, o semáforo, e o vetor de veículos. Antes de iniciar a execução, todos esses parâmetros são validados para evitar acessos inválidos à memória.

Durante cada iteração do laço principal, correspondente ao tick N-1 da simulação, a thread executa as seguintes etapas:

1. Obtém o tick atual por meio da função *clock_get_tick()*.
2. Limpa completamente o buffer interno do renderizador, preenchendo-o com espaços e inserindo as quebras de linha necessárias.
3. Reconstrói o cenário desenhando todos os tiles do mapa no buffer, utilizando os assets previamente carregados para cada tipo de célula.

4. Sobreposição dos veículos ativos sobre o mapa, desenhando o asset correspondente ao tipo e à direção de cada veículo em sua posição atual.
5. Limpa o terminal e imprime o conteúdo completo do buffer, exibindo também o número do tick atual.
6. Chama `clock_signal()` para informar ao relógio que a renderização daquele tick foi concluída, permitindo o avanço sincronizado da simulação.

Diferentemente de uma renderização incremental, esta implementação realiza um redesenho completo do mapa a cada tick. Embora essa abordagem seja menos eficiente em termos de processamento, ela simplifica a lógica de atualização da tela e evita inconsistências visuais, como artefatos deixados por veículos que mudaram de posição.

O renderizador utiliza um buffer linear de caracteres para montar toda a imagem antes de enviá-la ao terminal. Essa estratégia reduz o efeito de cintilação, pois a tela é atualizada apenas após toda a representação gráfica estar pronta.

2.3. Mecanismos de Sincronização

2.3.1. Sincronização pelo Clock

O módulo `Clock` funciona como coordenador do tempo da simulação. Cada thread possui uma participação na barreira de sincronização, incluindo:

- Threads dos veículos;
- Thread do analisador;
- Thread do renderizador;
- Thread do semáforo;
- Outros componentes sincronizados da simulação.

Ao concluir suas operações em um determinado tick, a thread sinaliza sua conclusão ao relógio. O avanço para o próximo tick ocorre somente quando todos os participantes esperados finalizaram suas tarefas.

O fluxo geral é:

1. Thread executa tarefa do tick atual
2. Finaliza processamento
3. Chama `clock_signal()`
4. Clock registra conclusão
5. Aguarda demais threads
6. Libera próximo tick

Dessa forma, nenhuma thread precisa ficar verificando continuamente se as outras terminaram.

2.3.2 Sincronização entre Threads

Cada thread possui um ciclo semelhante:

Veículos

1. Executa movimento
2. Envia resultado ao Clock
3. Aguarda próximo tick

Renderizador

1. Renderiza estado anterior
2. Sinaliza conclusão ao Clock
3. Aguarda próximo tick

Analizador

1. Processa requisições
2. Sinaliza conclusão ao Clock
3. Aguarda próximo tick

Nenhuma dessas threads precisa manter um laço consumindo CPU enquanto espera.

2.4. Ausência de espera ocupada

A nossa simulação evita o **uso de espera ocupada** através de mecanismos de sincronização entre threads, utilizando principalmente barreiras baseadas em condição de execução e comunicação entre os módulos por meio do Clock.

Na implementação, as threads não ficam verificando constantemente o estado da simulação. Cada componente executa sua tarefa apenas quando necessário e, ao finalizar o seu processamento referente ao tick atual, informa sua conclusão ao relógio global utilizando a função:

```
clock_signal(clock, current_tick);
```

Esse mecanismo permite que o relógio controle o avanço da simulação e mantenha todas as threads sincronizadas.

2.4.1. Espera por Eventos em vez de Verificação Contínua

A comunicação entre veículos e o analisador também evita espera ocupada. Quando um veículo deseja se movimentar, ele não tenta alterar diretamente o mapa nem fica verificando repetidamente se sua solicitação foi aceita. O processo ocorre através do módulo Analyser:

```
analyser_request(analyser, id, request);  
RequestStatus verdict = analyser_get_status(analyser, id);
```

O veículo envia sua solicitação e aguarda a decisão do analisador, permitindo que a lógica de autorização fique centralizada. Esse modelo evita que cada veículo execute continuamente verificações.

2.4.2. Uso de Mutex para Controle de Concorrência

Além da sincronização temporal, o sistema utiliza exclusão mútua para evitar conflitos no acesso a estruturas compartilhadas. Um exemplo ocorre no controlador de semáforos em:

```
pthread_mutex_lock(&traffic_light->mutex);  
/* atualização dos estados */  
pthread_mutex_unlock(&traffic_light->mutex);
```

O mutex garante que apenas uma thread modifique o estado dos semáforos de cada vez, evitando condições de corrida sem a necessidade de realizar verificações contínuas.

2.5. Estratégias contra deadlock

2.5.1. Centralização das decisões no Analyser

Na nossa simulação, os veículos não alteram diretamente o mapa. Em vez disso, todos enviam uma requisição ao Analyser e aguardam sua decisão.

```
// Monta a requisição e envia ao Analisador
const MovementRequest request = {
    .from = current_position,
    .to = target_position,
    .status = REQUEST_PENDING,
};

// Bloqueia a thread do veículo até que o Analisador decida por todos
analyser_request(analyser, id, request);
const RequestStatus verdict = analyser_get_status(analyser, id);
```

O principal mecanismo de prevenção de deadlock que usamos centraliza todas as decisões de movimentação no módulo **Analyser**. Em vez de cada thread tentar reservar ou bloquear diretamente as posições do mapa, todos os veículos apenas enviam uma requisição contendo sua posição atual e o destino pretendido.

Após receber todas as solicitações referentes ao tick atual, o Analyser resolve os conflitos de maneira centralizada, aprovando ou rejeitando cada movimento. Como somente um componente toma as decisões, elimina-se a

possibilidade de duas threads aguardar indefinidamente por recursos ocupados uma pela outra.

2.5.2. Uso de um único mutex para recursos compartilhados

A prevenção de deadlock está implementada em *update_vehicle()* em *src/vehicle.c* e decorre de *map_transfer_occupant* usar um único mutex global, não de ordenação de locks por endereço de memória. Isso significa que quando uma movimentação precisa modificar o estado do mapa, a operação é realizada utilizando um único mutex global responsável pela proteção da estrutura compartilhada.

Essa estratégia impede que uma thread adquira vários mutexes simultaneamente, eliminando a possibilidade de **espera circular**, uma das quatro condições necessárias para ocorrência de deadlock.

Como cada operação crítica utiliza apenas um bloqueio por vez, não existe dependência entre múltiplos recursos.

2.5.3. Sincronização por ticks

Outro mecanismo importante é a sincronização promovida pelo Clock por meio de:

```
clock_signal(clock, current_tick);  
size_t current_tick = clock_get_tick(clock);
```

Todas as threads executam somente uma etapa da simulação por tick e somente prosseguem após a sincronização realizada pelo relógio global.

Esse modelo evita que algumas threads avancem enquanto outras ainda manipulam estruturas compartilhadas, e isso reduz a possibilidade de bloqueios entre operações concorrentes.

2.5.4. Validações antes do acesso ao recurso compartilhado

Antes de solicitar qualquer movimentação ao analisador, cada veículo realiza diversas verificações locais, como é possível verificar abaixo:

```
if (!map_is_within_bounds(map, intended_target)) { ... }  
else if (!is_adjacent(map, vehicle, intended_target)) { ... }  
else if (is_overtaking(map, vehicle, intended_target)) { ... }  
else { target_position = intended_target; }
```

Essas verificações detectam movimentos inválidos antes que eles sejam enviados ao Analyser e diminuem o número de conflitos concorrentes que precisam ser resolvidos pelo sistema.

2.6. Ambulância

O sistema de prioridade da ambulância permite que, durante a simulação, os semáforos concedam passagem preferencial ao veículo de emergência, reduzindo o tempo de espera em cruzamentos e simulando o comportamento esperado no trânsito real.

A prioridade foi implementada através da comunicação entre o módulo dos veículos *vehicle.c* e o controlador global de semáforos *traffic_light.c*.

O mecanismo funciona em quatro etapas:

1. A ambulância identifica sua posição e direção de deslocamento.
2. Essa informação é registrada como coordenada de prioridade.
3. O controlador de semáforos consulta essa coordenada a cada tick da simulação.
4. Os semáforos pertencentes ao trajeto da ambulância recebem sinal verde, sobrepondo temporariamente o ciclo normal.

2.6.1. Controle da Coordenada de Prioridade

O módulo dos veículos possui uma coordenada global compartilhada que representa a prioridade ativa da ambulância.

```
Coord g_priority_coord = NULL_COORD;  
pthread_mutex_t g_priority_mutex = PTHREAD_MUTEX_INITIALIZER;
```

Essa variável é protegida por um mutex para evitar **condições de corrida** entre a thread da ambulância e a thread responsável pelos semáforos.

2.6.2. Atualização da Prioridade

Sempre que a ambulância realiza um movimento aprovado, sua coordenada de prioridade é atualizada pela função:

```
set_priority_coord(vehicle);
```

Essa função verifica se o veículo é uma ambulância. Caso não seja, nenhuma prioridade é registrada. Depois disso, o sistema determina qual eixo deve receber prioridade.

- Se a ambulância estiver trafegando horizontalmente (esquerda ou direita), é armazenada sua coordenada **Y**.
- Se estiver trafegando verticalmente (cima ou baixo), é armazenada sua coordenada **X**.

Assim, toda a rua que a ambulância estiver percorrendo pode ser identificada pelo controlador de semáforos.

2.6.3. Consulta da Prioridade

A thread responsável pelos semáforos consulta a prioridade ativa a cada tick através da função:

```
vehicle_get_priority_coord();
```

Ela também utiliza o mutex compartilhado para garantir consistência entre leitura e escrita.

2.6.4. Integração com o Controlador de Semáforos

Durante cada ciclo da thread dos semáforos ocorre a sequência:

```
const Coord priority = vehicle_get_priority_coord();  
  
update_targets(traffic_light, priority);  
  
update_cycle(traffic_light);
```

Primeiro a coordenada da ambulância é capturada e logo após, enviada para *update_targets()*, que recalcula quais semáforos deverão permanecer verdes ou vermelhos naquele tick. Por fim, a máquina de estados realiza as transições necessárias.

2.6.5. Sobrescrição do Ciclo Normal

De início, a função *update_targets()* calcula os estados considerando apenas o eixo aberto da simulação. Após isso, ocorre uma segunda etapa denominada *Sobreposição de Prioridade*, responsável por alterar temporariamente esse comportamento. Com ela, o algoritmo verifica se existe uma coordenada válida de prioridade. Quando existe, ele percorre todos os semáforos cadastrados.

Obs.: Sempre que um semáforo pertence à mesma linha ou coluna da ambulância, seu estado alvo é alterado para verde:

```
light->target = TRAFFIC_LIGHT_GREEN;
```

Assim, mesmo que o ciclo normal determinasse vermelho, o semáforo recebe prioridade para permitir a passagem da ambulância.

2.6.6. Segurança da Implementação

A implementação da ambulância inclui mecanismos para garantir funcionamento seguro em ambiente concorrente que são:

- A utilização de *pthread_mutex* para proteger a coordenada global de prioridade;
- Leitura e escrita sincronizadas entre as threads da ambulância e dos semáforos;
- Atualização da prioridade apenas quando o movimento da ambulância é aprovado pelo analisador de movimentação;
- Manutenção da máquina de estados dos semáforos, preservando as transições de cor, evitando mudanças bruscas de sinalização.

3. Membros e responsabilidades

Membro	Responsabilidade
Gabriel Souza Santos	Implementação do clock, definição de estrutura do projeto, renderização, debug e revisão do código.
José Dhonatan Fernandes de Almeida	Criação do mapa, funções auxiliares de veículo, implementação do semáforo sem espera ocupada.
Letícia Maria dos Santos Dias	Implementação dos veículos normais, issues e divisão de responsabilidades, relatório técnico e documentação.
Sarah Mendes Teles	Implementação da ambulância, sistema de prioridade e mecanismos para prevenção de deadlocks e readme.

4. Conclusão

Em suma, com o presente trabalho conseguimos desenvolver uma simulação que refletisse o funcionamento das threads em sistemas operacionais de maneira

metafórica considerando o contexto de tráfego de veículos de modo a consolidar nossos aprendizados na disciplina de Sistemas Operacionais. Todos os arquivos individuais presentes no código foram detalhadamente documentados por meio das docstrings no próprio código. Para testar a nossa solução, acesse: [github - sistema de tráfego urbano](#), clone o repositório e execute as instruções do readme.

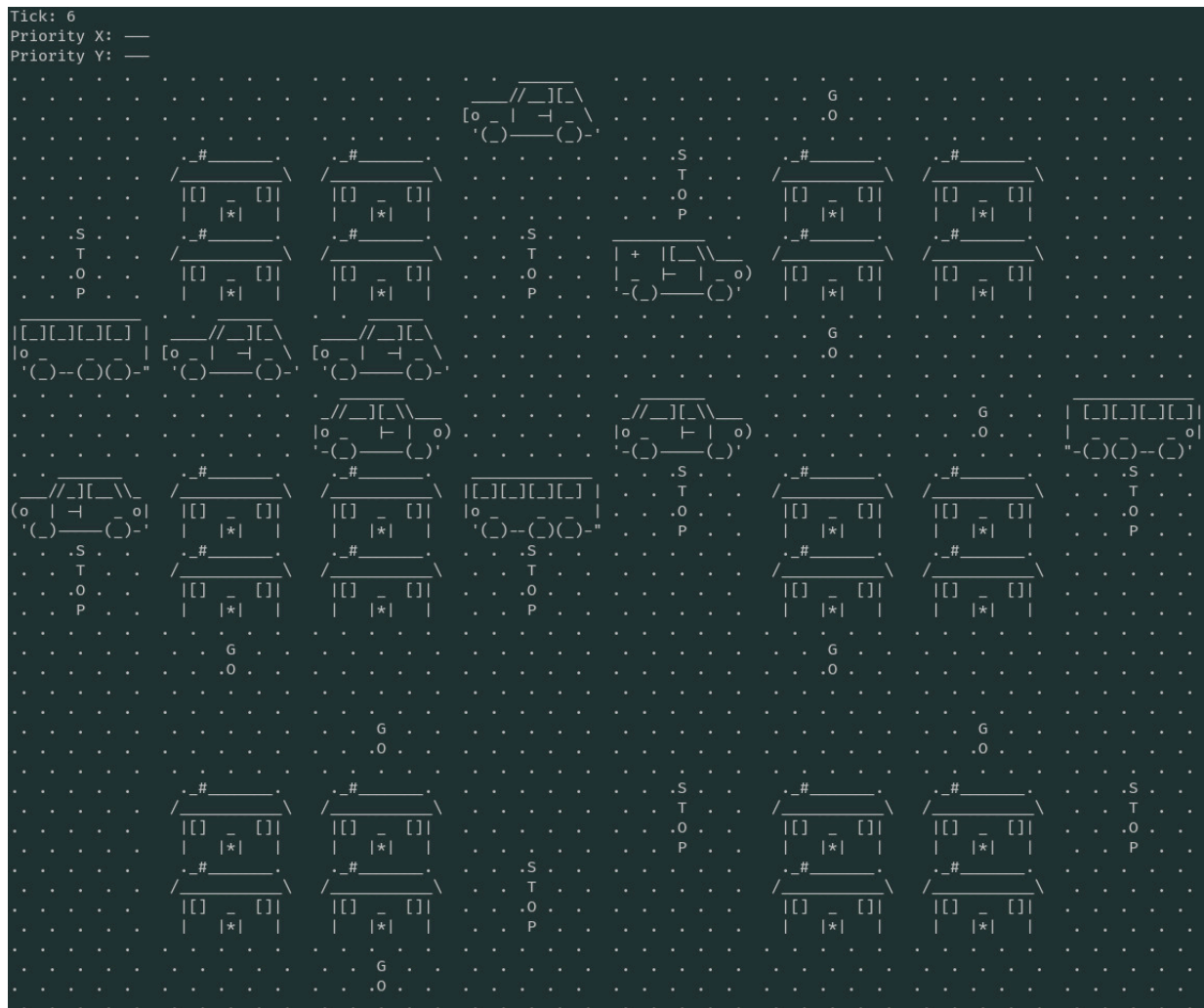


Imagem do sistema em funcionamento. Artes retiradas e adaptadas de <https://www.asciart.eu/>

Obs.: como não tínhamos imagens dos veículos na vertical, eles aparecem sempre no formato horizontal.